

DevOps Hackathon – Python and Shell Scripting

Mahesh Dineshkumar(UST,IN) Room: 3

Shell Scripting

Scenario 1:

Problem Statement

Operations team requires quick health summary.

Objective

Build a script that reports:

- CPU usage
- Memory usage
- Disk usage
- Top processes
- Active network connections

Generate a structured summary report.

Solution:

I created a shell script in Amazon Linux running on an EC2 instance in Amazon Web Services to generate a structured system health summary report for the operations team.

- First, I launched an EC2 instance using Amazon Linux.
- Then I connected to the instance using EC2 Instance Connect.
- Created a shell script with name **“health_report.sh”**.
- Generated a Time stamped report file to ensure that every time this gets executed, it will have a new file with the time and date it has been executed.
 - **REPORT="health_report_\$(date +%F_%H-%M-%S).txt"**
- Collected system metrics using native Linux commands, **“top”** for CPU usage, **“free -m”** for memory usage, **“df -h”** for disk utilization, **“ps aux --sort=-%cpu”** to fetch top resource-consuming processes, and **“ss -tunap”** to list active network connections.
- Made the script executable using the command
 - **“chmod +x health_report.sh”** and executed using **“./health_report.sh”**.

Script inside the “health_report.sh”:

```
#!/bin/bash
```

```
REPORT="health_report_$(date +%F_%H-%M-%S).txt"
```

```
echo "===== " >> $REPORT
```

```
echo "    SYSTEM HEALTH REPORT    " >> $REPORT
```

```
echo "Generated on: $(date)" >> $REPORT
```

```
echo "===== " >> $REPORT
```

```
echo "" >> $REPORT
```

```
echo "CPU Usage:" >> $REPORT
```

```
top -bn1 | grep "Cpu(s)" >> $REPORT
```

```
echo "" >> $REPORT
```

```
echo "Memory Usage:" >> $REPORT
```

```
free -m >> $REPORT
```

```
echo "" >> $REPORT
```

```
echo "Disk Usage:" >> $REPORT
```

```
df -h >> $REPORT
```

```
echo "" >> $REPORT
```

```
echo "Top 5 CPU Processes:" >> $REPORT
```

```
ps aux --sort=-%cpu | head -6 >> $REPORT
```

```
echo "" >> $REPORT
```

```
echo "Active Network Connections:" >> $REPORT
```

```
ss -tunap >> $REPORT
```

Screenshots:

```
[ec2-user@ip-172-31-32-163 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-32-163 ~]$ nano health_report.sh
[ec2-user@ip-172-31-32-163 ~]$ chmod +x health_report.sh
[ec2-user@ip-172-31-32-163 ~]$ ./health_report.sh
[ec2-user@ip-172-31-32-163 ~]$ ls
health_report.sh  health_report  2026-02-18  09-13-42.txt
```

Report:

```
[ec2-user@ip-172-31-32-163 ~]$ cat health_report_2026-02-18_09-13-42.txt
=====
                SYSTEM HEALTH REPORT
Generated on: Wed Feb 18 09:13:42 UTC 2026
=====

CPU Usage:
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st

Memory Usage:
                total          used          free          shared  buff/cache          available
Mem:              916             171             522              0             223             614
Swap:               0               0               0

Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0  4.0M   0% /dev
tmpfs           459M   0  459M   0% /dev/shm
tmpfs           184M 424K  183M   1% /run
/dev/nvme0n1p1  8.0G 1.6G  6.4G  20% /
tmpfs           459M   0  459M   0% /tmp
/dev/nvme0n1p28  10M 1.3M  8.7M  13% /boot/efi
tmpfs           92M   0   92M   0% /run/user/1000
```

```
Top 5 CPU Processes:
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  1.8 172420 17316 ?        Ss   08:56   0:00 /usr/lib/systemd/systemd --switched-root --system --deserialize=32
root         2  0.0  0.0      0     0 ?        S   08:56   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        I<   08:56   0:00 [rcu_gp]
root         4  0.0  0.0      0     0 ?        I<   08:56   0:00 [rcu_par_gp]
root         5  0.0  0.0      0     0 ?        I<   08:56   0:00 [slub_flushwq]

Active Network Connections:
Netid State  Recv-Q Send-Q           Local Address:Port      Peer Address:Port Process
udp    UNCONN  0      0                127.0.0.1:323          0.0.0.0:*
udp    UNCONN  0      0          172.31.32.163%ens5:68 0.0.0.0:*
udp    UNCONN  0      0                [::1]:323              [::]:*
udp    UNCONN  0      0 [fe80::c0:99ff:fecf:225f]%ens5:546 [::]:*
tcp    LISTEN  0     128                0.0.0.0:22             0.0.0.0:*
tcp    ESTAB   0      0          172.31.32.163:22       13.233.177.4:19801
tcp    TIME-WAIT 0      0          172.31.32.163%ens5:50604 169.254.169.254:80
tcp    TIME-WAIT 0      0          172.31.32.163%ens5:50616 169.254.169.254:80
tcp    TIME-WAIT 0      0          172.31.32.163%ens5:50572 169.254.169.254:80
tcp    LISTEN  0     128                [::]:22                [::]:*
```

Scenario 2:

Problem Statement:

Application logs grow indefinitely.

Objective

- Build a shell-based log rotation tool that:
- Compresses old logs
- Deletes logs older than X days
- Maintains rotation history
- Logs all operations

Solution:

I created a shell script named **“log_rotate.sh”** that performs the following operations:

1. Created the script with defined directories: **“LOG_DIR”**, **“ARCHIVE_DIR”**, **“DAYS”**, **“HISTORY_FILE”**.
2. Initially, while writing the **“for”** loop to iterate through. **“Log”** files, I forgot to include the **“do”** keyword after the loop declaration, which caused a syntax error when executing the script. After reviewing the loop structure, I corrected it.
3. Used the corrected **“for”** loop to compress. **“Log”** files using the **“gzip”** command and moved them to the archive directory.
4. During execution, I also faced an issue where the script was not producing any visible output. After debugging, I realized that the required log directory did not exist and there were no **“log”** files to process. I resolved this by manually creating the necessary directories and test log files.
5. Logged all operations such as compression, deletion, and execution time into a rotation history file to maintain proper tracking.
6. Finally, made the script executable using **“chmod +x log_rotate.sh”** and executed it using **“./log_rotate.sh.”**

Screenshots:

```
[ec2-user@ip-172-31-32-163 ~]$ nano log_rotate.sh
[ec2-user@ip-172-31-32-163 ~]$ ls
health_report.sh  health_report_2026-02-18_09-13-42.txt  log_rotate.sh
[ec2-user@ip-172-31-32-163 ~]$ chmod +x log_rotate.sh
[ec2-user@ip-172-31-32-163 ~]$ ./log_rotate.sh
./log_rotate.sh: line 14: syntax error near unexpected token `if'
./log_rotate.sh: line 14: `    if [ -f "$file" ]; then'
[ec2-user@ip-172-31-32-163 ~]$
```

This is where I faced the syntax issue: didn't add **“do”** inside for loop

```
[ec2-user@ip-172-31-32-163 ~]$ ./log_rotate.sh
[ec2-user@ip-172-31-32-163 ~]$ ls /home/ec2-user/app_logs
[ec2-user@ip-172-31-32-163 ~]$ touch /home/ec2-user/app_logs/test1.log
[ec2-user@ip-172-31-32-163 ~]$ touch /home/ec2-user/app_logs/test2.log
[ec2-user@ip-172-31-32-163 ~]$ ls /home/ec2-user/app_logs
test1.log  test2.log
[ec2-user@ip-172-31-32-163 ~]$ ./log_rotate.sh
[ec2-user@ip-172-31-32-163 ~]$ ls /home/ec2-user/log_archive
test1.log.gz  test2.log.gz
[ec2-user@ip-172-31-32-163 ~]$ cat rotation_history.log
Log Rotation Started at Wed Feb 18 10:04:45 UTC 2026
Log Rotation Started at Wed Feb 18 10:08:48 UTC 2026
Deleted logs older than 7 days
Log Rotation Completed at Wed Feb 18 10:08:48 UTC 2026
Log Rotation Started at Wed Feb 18 10:08:59 UTC 2026
Deleted logs older than 7 days
Log Rotation Completed at Wed Feb 18 10:08:59 UTC 2026
Log Rotation Started at Wed Feb 18 10:13:15 UTC 2026
Deleted logs older than 7 days
Log Rotation Completed at Wed Feb 18 10:13:15 UTC 2026
Log Rotation Started at Wed Feb 18 10:14:38 UTC 2026
Compressed and moved /home/ec2-user/app_logs/test1.log
Compressed and moved /home/ec2-user/app_logs/test2.log
Deleted logs older than 7 days
Log Rotation Completed at Wed Feb 18 10:14:38 UTC 2026
[ec2-user@ip-172-31-32-163 ~]$
```

And here I manually created the necessary directories and test log files.

Python Scripting

Scenario 3:

Problem Statement

Production logs do not always contain explicit “ERROR” keywords.

Objective

Build a tool that:

- Analyzes word frequency in logs
- Identifies rare or unusual patterns
- Flags potential anomalies
- Produces anomaly insights summary

Solution:

I developed a Python-based log anomaly detection tool using Visual Studio Code on Windows to analyze production logs that do not always contain explicit “ERROR” keywords.

- First, I created a project folder named LogAnalyzer inside my Practice directory.
- Created a Python virtual environment (venv) to maintain an isolated development setup.
- Wrote a Python script named “**log_analyzer.py**” inside the LogAnalyzer folder.
- Created a log file named “**app.log**” containing sample production log entries.
- Implemented functionality in the script to:
 - Read log entries from the log file
 - Analyze word frequency using Python’s collections module
 - Identify rare words (frequency ≤ 1)
 - Flag log lines containing rare words as potential anomalies
 - Generate a structured anomaly summary
- Executed the script using the virtual environment Python interpreter:

Script:

```
import collections

LOG_FILE = "app.log"
def read_logs(file_path):
    with open(file_path, "r") as f:
        return f.readlines()

def word_frequency(lines):
    words = []
    for line in lines:
        words.extend(line.strip().lower().split())
    return collections.Counter(words)

def find_rare_words(freq_dict, threshold=1):
    rare = []
    for word, count in freq_dict.items():
        if count <= threshold:
            rare.append(word)
    return rare

def detect_anomalies(lines, rare_words):
    anomalies = []
    for line in lines:
        for word in rare_words:
            if word in line.lower():
                anomalies.append(line.strip())
                break
    return anomalies

def main():
    lines = read_logs(LOG_FILE)
    freq = word_frequency(lines)
    rare_words = find_rare_words(freq, threshold=1)
    anomalies = detect_anomalies(lines, rare_words)

    print("==== ANOMALY SUMMARY =====")
    print(f"Total log lines: {len(lines)}")
    print(f"Unique words: {len(freq)}")
    print(f"Rare words detected: {len(rare_words)}")
    print(f"Potential anomalies found: {len(anomalies)}\n")
    print("---- Suspicious Log Entries ----")
    for entry in anomalies:
        print(entry)

if __name__ == "__main__":
    main()
```

Screenshots:

```
===== ANOMALY SUMMARY =====  
Total log lines: 9  
Unique words: 22  
Rare words detected: 15  
Potential anomalies found: 7  
  
---- Suspicious Log Entries ----  
User login successful  
Database connection established  
Unexpected token received  
Memory allocation failure detected  
User logout successful  
Database timeout occurred  
Random unusual activity detected
```